

point. Possible values are top, hanging, middle, alphabetic, ideographic, or bottom.

As you might expect, the text drawing routines consist of two functions on the context object:

1. `fillText(text, x, y, maxwidth)`
2. `strokeText(text, x, y, maxwidth)`

We can now draw the text we need:

```

• context.font = "bold 12px sans-serif";
• context.fillText("x", 248, 43);
• context.fillText("y", 58, 165);
•
• context.textBaseline = "top";
• context.fillText("( 0 , 0 )", 8, 5);
•
• context.textAlign = "right";
• context.textBaseline = "bottom";
• context.fillText("( 500 , 375 )", 492, 370);
•
• context.fillRect(0, 0, 3, 3);
• context.fillRect(497, 372, 3, 3);
•

```

Let's see what we have done here:

The first three lines are easy to understand. We have specified the font, the font size and font style and then we have written the letters 'x' and 'y'.

For the text in the upper-left corner, we have put the top of the text to be at `y=5`. Instead of measuring the height of the text and calculating the baseline, we have set `textBaseline` to `top` and passed the upper-left coordinate of the text's bounding box.

For the text in the lower-right corner, we have put the text to be at coordinates (492,370). Instead of measuring the height and the width of the text, we have set `textAlign` to `right` and `textBaseline` to `bottom`, and then called `fillText()` with the bottom-right coordinates of the text's bounding box.

The last two lines add two small rectangles and fill it with the default black color. Voila! We have our final coordinate system.

Now let us take a look at some other cool stuff that you can do with Canvas

Inserting Images